# 校易淘 · 校园二手交易系统

验 收 材 料 汇 编

代码基线：main 分支
技术栈：Vue 3 + Spring Boot 4.1.0 + MySQL + Redis
编制日期：2026 年 10 月

## 一、项目与技术栈概览

校易淘是一个前后端完整打通的校园二手交易系统：前端 Vue 3 单页应用，后端基于若依（RuoYi）Spring Boot 扩展业务模块，数据层 MySQL + Redis。买家与卖家的交易闭环（浏览、收藏、购物车、下单、支付、发货、收货、退款）均已实现并通过端到端验证。

### 1.1 技术栈总表

| 类别 | 技术 | 版本 | 说明 |
|---|---|---|---|
| 编程语言 | Java / JavaScript / TypeScript / SQL | Java 17 编译<br>JDK 27 运行 | — |
| 开发平台 | Windows 11 • Maven • Node.js • Git | 3.9.9 / 24.21.0 / 2.55.0 | — |
| 后端框架 | Spring Boot | 4.1.0 | 若依 3.9.2 |
| 持久层 | MyBatis / PageHelper | 4.1.0 / 4.1.1 | 预编译防注入 |
| 数据库 | MySQL + Flyway | 26.7.0 / 12.4.0 | 结构版本化 |
| 连接池 | Druid | 1.2.28 | 带监控台 |
| 缓存 | Redis | 7.4.3 | 令牌 / 限流计数 |
| 前端框架 | Vue + Element Plus + Vite | 3.5.44 / 2.5.6 / 6.4.1 | 单页应用 |
| 鉴权 | JWT（JJWT） | 0.9.1 | 访问 + 刷新令牌 |

| | | | |
|---|---|---|---|
| 可观测 | Actuator + Micrometer Prometheus | 随 Boot 4.1.0 | /actuator/prometheus |
| 接口文档 | SpringDoc（OpenAPI 3） | 3.1.0 | Swagger UI |
| 容器 / CI | Docker Compose / GitHub Actions | — | 四服务、三作业 |
| 测试 | JUnit 5 / Mockito / node:test / JaCoCo | 0.8.15 | 38 + 19 用例 |



图 1　运行环境与版本（本机实际执行输出）

# 二、架构讲解

## 2.1 分层与边界

系统是**单体 + 分层**，没有拆微服务。理由很直接：交易、库存、支付落在同一个库里，拆开就要处理分布式事务，收益抵不上复杂度。分层的目的是把「谁能碰数据库、谁只做编排」定死。

| 模块 / 层 | 职责 | 不该做的事 |
| --- | --- | --- |
| ruoyi-admin | 启动入口、全局配置、打包、Actuator | 写业务规则 |
| ruoyi-framework | 安全（JWT/RBAC/限流/XSS）、拦截器、TraceId、统一异常 | 写交易逻辑 |
| ruoyi-campus | 业务：商品、订单、支付、退款的状态机与校验 | 拼 SQL 字符串 |
| mapper XML | SQL 与索引；原子扣减只放这里 | 做业务判断 |
| Vue 3 SPA | 展示与交互，只通过 /api/v1 访问 | 信任前端传来的身份与金额 |

## 2.2 一次下单请求的完整路径

前端 `POST /api/v1/orders` → **JwtAuthenticationTokenFilter** 解析 Bearer 令牌写入安全上下文 → **RateLimitInterceptor** 按 IP 计数（超限直接 429）→ **Controller** 校验请求体 → **Service** 开事务：校验归属与商品状态、建订单与明细、**原子扣减库存**、写幂等记录 → 提交返回订单号。任一步失败整体回滚，库存不会被扣。

> 为什么状态机在 Service 而不是前端：订单有 7 个状态，非法跃迁（未付款就发货）必须服务端拦。按钮只是提示，接口可以被直接调用。

## 2.3 部署架构



图 2　部署架构：接入层 / 应用层 / 数据层 / 可观测 / CI-CD

| 形态 | 接入方式 | 说明 |
| --- | --- | --- |
| 开发态 | Vite dev server（:5174）代理 /api 到 :8080 | 热更新 |
| 容器态 | Nginx 反代 /api，仅前端端口对外 | compose 四服务 |
| HTTPS | Spring Boot 内置 TLS，:8443 | 自签证书验证；生产建议 Nginx 终止 |

# 三、数据库与缓存

数据库 shop 共 **45 张表**，业务表 13 张（ct_ 前缀），其余为若依系统表与 Quartz 调度表。表结构由 Flyway 管理，flyway_schema_history 记录 V1（存量基线）与 V2（订单索引），启动时自动校验迁移。



图 3　MySQL 26.7.0 实查：版本、业务表、数据量、Flyway 迁移记录

## 3.1 关键索引

| 索引 | 表 | 作用 |
| --- | --- | --- |
| uk_ct_order_no | ct_order | 订单号唯一 |
| uk_ct_order_idem | ct_order | 幂等键唯一，防重复下单 |
| uk_ct_order_group_seller | ct_order | 多卖家拆单分组唯一 |
| uk_ct_idem_key | ct_idempotency | 幂等记录唯一约束 |
| uk_ct_favorite (user_id, product_id) | ct_favorite | 防重复收藏 |
| uk_ct_cart (user_id, product_id) | ct_cart_item | 同商品合并 |
| uk_ct_payment_no / uk_ct_refund_no | ct_payment / ct_refund | 支付单、退款单唯一 |

# 四、企业级能力对照

> 下表只列代码中真实存在、可点开验证的能力，并给出位置；没做到的以「未覆盖」标注，不做包装。

## 4.1 安全

| 能力 | 状态 | 实现与证据 |
| --- | --- | --- |
| OAuth2 / JWT | 已覆盖 | JJWT 0.9.1；TokenService 签发校验；JwtAuthenticationTokenFilter 解析 Bearer；刷新令牌落 Redis |
| RBAC | 已覆盖 | sys_user / sys_role / sys_menu / sys_role_menu + @PreAuthorize |
| 密码哈希 | 已覆盖 | BCryptPasswordEncoder，库中不存明文 |
| HTTPS | 已覆盖 | application-https.yml + 自签证书，8443 实测 200（见第六章） |
| 防越权 | 已覆盖 | CtProductServiceImpl:240；订单按 buyer/seller 校验；越权按 404 处理不泄露存在性 |
| 防 SQL 注入 | 已覆盖 | MyBatis 预编译 #{}，mapper 中无 ${} 拼接用户输入 |
| XSS | 已覆盖 | XssFilter:22 + XssHttpServletRequestWrapper，FilterConfig:14 注册 |
| CSRF | 已覆盖 | SecurityConfig:90,93 关闭——无状态 JWT，无 Cookie 会话攻击面 |
| 接口限流 | 已覆盖 | RateLimit 注解 + 滑动窗口；内存 / Redis 双实现，多实例用 ZSET+Lua |
| 敏感数据脱敏 | 部分 | 日志不打印密码；未做统一手机号脱敏 |
| 审计日志 | 已覆盖 | 业务接口 43 处 @Log，写入 sys_oper_log |

## 4.2 数据一致性

| 能力 | 状态 | 实现与证据 |
| --- | --- | --- |
| 事务 | 已覆盖 | campus 14 处 @Transactional(rollbackFor=Exception)：CtOrderServiceImpl:120,327 等 |
| 索引 | 已覆盖 | 见 3.1 |
| 连接池 | 已覆盖 | Druid max-active=8，带 SQL 监控 |
| 分页 | 已覆盖 | PageHelper 4.1.1 |
| 库存防超卖 | 已覆盖 | deductStock：set stock = stock - #{qty} ... where stock >= #{qty}；StockConcurrencyTest 并发验证 |
| 幂等 | 已覆盖 | ct_idempotency + 唯一键；ApiOrderController:37-50 支持 idempotencyKey；IdempotencyConcurrencyTest 验证 |
| 分布式事务 | 未采用 | 单体单库，本地事务已足够，不引入无用框架 |

## 4.3 性能

| 能力 | 状态 | 实现与证据 |
| --- | --- | --- |
| Redis 缓存 | 已覆盖 | 7.4.3，承载刷新令牌、登录态、限流计数 |
| 异步处理 | 未采用 | 无 @Async，强一致优先于吞吐 |
| 消息队列 | 未采用 | 单体同步事务；幂等表已覆盖重复投递 |
| CDN | 未采用 | 静态资源由 Nginx 提供 |
| 连接池 | 已覆盖 | 同 4.2 |

| | | |
|---|---|---|
| 压测报告 | 已覆盖 | 见第五章：并发 50 时 1354 QPS、p95 56.8ms、0% 错误 |

### 4.4 可靠性

| 能力 | 状态 | 实现与证据 |
|---|---|---|
| 日志 | 已覆盖 | Logback 分级：sys-info / sys-error / api-access，每条带 trace |
| 监控 | 已覆盖 | Druid 监控台 /druid；ApiTraceFilter 生成 X-Trace-Id 写入 MDC |
| 指标与告警 | 已覆盖 | /actuator/prometheus 暴露 QPS/RT/JVM/堆/磁盘；5 条告警规则见 docs/ops/alert-rules.yml |
| 健康检查 | 已覆盖 | GET /api/v1/system/health 与 /actuator/health（含 DB 组件状态） |
| 熔断 / 降级 | 未覆盖 | 未引入 Resilience4j；限流器对 Redis 故障做了 fail-open |
| 重试 | 有限 | 前端 http.js 对 401 刷新令牌后重试一次 |
| 备份 / 恢复 | 已覆盖 | scripts/backup.ps1 + restore.ps1；实测往返：106KB dump 恢复后 45 表 / 8 商品 / 2 订单一致 |

### 4.5 工程化

| 能力 | 状态 | 实现与证据 |
|---|---|---|
| Git 规范 | 简化 | 主干开发，提交遵循 Conventional Commits |
| 代码规范 | 约定 | 遵循若依分层命名；未接 Checkstyle |
| 静态检查 | 未覆盖 | 无 Checkstyle / ESLint |
| 单元测试 | 已覆盖 | 后端 38 例（JUnit 5 + Mockito），前端 19 例（node:test） |
| 集成测试 | 已覆盖 | 订单全链路、库存并发、幂等并发、限流共享、支付网关、刷新令牌、可观测性、契约 |
| E2E | 未覆盖 | 以 compose 冒烟 + 57 个用例替代 |
| 覆盖率 | 已覆盖 | JaCoCo：业务模块行 43.2%、分支 34.6%；CI 有 35% 门禁 |
| OpenAPI | 已覆盖 | SpringDoc /v3/api-docs/campus；契约快照 + CI 漂移校验 |
| CI/CD | 已覆盖 | GitHub Actions 三作业 + 覆盖率门禁 + 依赖漏洞门禁 |
| Docker / 配置分离 | 已覆盖 | 前后端 Dockerfile + compose；yml + profile + 环境变量，凭证不落库 |

### 4.6 运维部署

| 能力 | 状态 | 实现与证据 |
|---|---|---|
| 环境区分 | 已覆盖 | 基线 yml + profile（druid / https）+ 环境变量覆盖；测试库独立准备 |
| 部署架构图 | 已覆盖 | 见 2.3 图 2 |
| 回滚方案 | 已覆盖 | 代码回退镜像 tag / git revert；数据 restore.ps1；结构用新迁移脚本 |
| 容器化 | 已覆盖 | docker compose up -d --build；CI 真实验证四服务起停与冒烟 |

### 4.7 未覆盖项汇总

| 项 | 原因 | 若补，最小改动 |
|---|---|---|
| 消息队列 / 分布式事务 | 单库单体，本地事务已够用 | 拆服务或支付异步化时再引入 |
| CDN | 本地演示不需要 | 静态资源接云 CDN |

| | | |
|---|---|---|
| 熔断 / 降级 | 单体内部调用，无限流以外的下游依赖 | 引入 Resilience4j 包一层外部调用 |
| 静态检查 / E2E | 时间取舍 | Checkstyle / ESLint / Playwright |
| 统一字段脱敏 | 仅做到不打印 | Jackson @JsonSerialize 脱敏 |

## 五、性能压测

压测工具为自研零依赖 Node.js 脚本（`scripts/loadtest.mjs`），固定并发 worker 循环发压，按权重混合 4 个读接口（健康探活 1 : 商品列表 4 : 类目 2 : 商品详情 2）。

| 并发 / 时长 | 请求数 | QPS | 错误率 | avg | p50 | p95 | p99 |
|---|---|---|---|---|---|---|---|
| 20 / 20s | 22418 | 1120.9 | 0.00% | 17.8ms | 16.7ms | 32.9ms | 49.0ms |
| 50 / 20s | 27081 | 1354.0 | 0.00% | 36.9ms | 38.8ms | 56.8ms | 68.4ms |

| 场景（并发 50） | QPS | avg(ms) | p95(ms) | 错误率 |
|---|---|---|---|---|
| 健康探活 | 150.3 | 4.0 | 6.4 | 0.00% |
| 类目列表 | 300.9 | 35.5 | 47.3 | 0.00% |
| 商品详情 | 300.8 | 35.0 | 47.4 | 0.00% |
| 商品列表（分页 20） | 602.0 | 46.8 | 61.5 | 0.00% |

> 两轮合计 49499 个请求全部 2xx。压测客户端与服务端同机运行、互相争抢 CPU，该数据为「同机自压」结果，不等同于独立压测机下的极限能力。

- 并发 20 → 50，QPS 由 1121 升到 1354（+21%），avg 由 17.8ms 升到 36.9ms，属「延迟换吞吐」的正常区间
- 商品列表是最重路径（avg 46.8ms），因其带分页与多次关联查询；健康检查仅 4ms，框架本身开销很小

# 六、测试与覆盖率

## 6.1 用例分布

| 层次 | 工具 | 用例数 | 覆盖内容 |
| --- | --- | --- | --- |
| 后端单元 / 集成 | JUnit 5 + Spring Boot Test + Mockito | 38 | 订单生命周期、库存并发、幂等并发、限流共享、支付网关、刷新令牌、可观测性、契约 |
| 前端单元 | Node.js 内置 node:test | 19 | HTTP 层、错误契约、令牌刷新、自动登出 |
| 接口契约 | openapi-typescript + 导出校验 | — | 运行时 OpenAPI 与仓库快照一致性 |

## 6.2 覆盖率（JaCoCo 0.8.15）

| 范围 | 指令 | 分支 | 行 | 方法 |
| --- | --- | --- | --- | --- |
| ruoyi-campus 业务模块 | 41.4% | 34.6% | 43.2% | 45.1% |
| 全后端（含框架模块） | 27.1% | 15.4% | 28.6% | 33.9% |

达到 100% 行覆盖的类：TraceContext、ApiNotFoundHandler、CampusApiContractConfig、SandboxPaymentGateway、ApiWebMvcConfig 等。

**门禁：**`scripts/check-coverage.ps1` 读取聚合报告，campus 行覆盖低于 35% 时构建失败，并列出未覆盖行最多的 10 个类。本地已验证正反两向：43.2% 通过、阈值提到 60% 时失败并给出清单。



图 4　JaCoCo 覆盖率报告

# 七、部署、可观测与高可用

## 7.0 多实例与负载均衡

单实例是最大的单点。多实例上线的真正门槛不是"能不能起两个进程"，而是**三个共享问题**：会话、限流、定时任务。这三件若还在进程内，起第二个副本只会制造新 bug。

| 共享依赖 | 单实例时 | 多实例后必须 |
| --- | --- | --- |
| 登录态 / 令牌 | 内存即可 | JWT 自校验 + 刷新令牌落 Redis（用户换节点不掉登录） |
| 接口限流 | 进程内滑动窗口 | Redis ZSET + Lua 原子计数（否则配额被放大 N 倍） |
| 定时任务 | RAMJobStore | JDBC JobStore 集群 + QRTZ_LOCKS 行锁（否则同一任务跑两遍） |

三项都已落地（见 7.3 与附录 B），因此补负载均衡只剩"加一层网关"。配置见 `docs/ops/nginx-lb.conf`，要点：`least_conn` 分发、`max_fails=2 fail_timeout=15s` 自动摘除、`keepalive 32` 复用长连接、透传 `X-Real-IP`（否则限流会把所有人算成同一个 IP 而互相连坐）、`proxy_next_upstream` 只对幂等读请求重试。

为了让"到底是哪台机器在服务"肉眼可查，后端加了 `InstanceIdFilter`，每个响应带 `X-Instance-Id`（取值 `CAMPUS_INSTANCE_ID` 或 主机名:端口）。`docker-compose.scale.yml` 与根 compose 叠加即可起两副本 + LB：

```
docker compose -f docker-compose.yml -f docker-compose.scale.yml up -d --build --scale backend=2 --wait
```

多实例 + 负载均衡：5 项断言全过

```
=== 多实例 + 负载均衡验证 ===
  LB=http://localhost:8088  nodes=8080,8082
[1] 流量是否真的分到两个节点
  [PASS] 两个节点都收到请求
        分布: node-a=4, node-b=4
[2] 令牌跨节点可用（无状态）
  [PASS] 同一令牌 6 次请求全部 200
  [PASS] 这 6 次确实落在两个不同节点上
[3] 限流配额全局共享（10 次/60 秒/IP）
  [PASS] 12 次里前 10 次放行、之后限流
[4] 单节点故障，服务不中断
        已杀掉 8082（pid=133668），等待 LB 摘除
  [PASS] 少一个节点后仍能正常服务
        仍在服务的节点: node-a
=== 结果: 5 通过 / 0 失败 ===
```

第 1 条证明流量真的分到两个节点；第 2 条证明令牌是无状态的（换节点不掉登录

图 5　多实例验证：分流、无状态令牌、共享限流、单节点故障共 5 项断言

四条断言的含义：① 8 次请求落到两个节点各 4 次；② 同一令牌连打 6 次（跨两节点）全部 200；③ 12 次刷新请求前 10 次放行、第 11 次起统一 429；④ 杀掉一个节点后 6 次请求全部成功。复现命令：`scripts\demo-lb.ps1 -KillNode 8082`。

**为什么第 3 条最关键：**如果限流仍是进程内计数，两个节点各放行 10 次，实际放行 20 次 —— 攻击者只要多打几次就能绕过一半的限制。这一条 429 从第 11 次开始而不是第 21 次，才说明配额是真共享的。

## 7.1 compose 服务

| 服务 | 镜像 | 说明 |
| --- | --- | --- |
| mysql | mysql:8.4 | 挂载 docker/mysql-init 自动导入种子数据 |
| redis | redis:7-alpine | 令牌、登录态、限流计数 |
| campus-backend | 自建 | 非 root 运行，分层构建 |
| campus-frontend | 自建 | 多阶段构建，Nginx 提供静态资源并反代 |

## 7.1b 日志集中化

多实例后日志分散在各节点磁盘上，「用户报错但不知道是哪台机器处理的那一次」只能一台台 grep。做法是应用直接往 Loki push（`LokiLogAppender`），而不是让 Promtail 抓文件 —— 容器里日志落在临时卷，抓文件要额外挂载；应用自己推则与部署形态无关，且能带上结构化字段（traceId / instance / level）。

| 设计点 | 为什么这么定 |
| --- | --- |
| 有界队列 + 满则丢最旧 | 日志不该把业务线程堵死，也不该把堆吃光；丢的是最老的日志，保住最近现场 |
| 推送失败不重试 | Loki 故障时重试会堆积成雪崩；只计数，每 30 秒打一行到 stderr 暴露问题 |
| 默认关闭 | 只有显式配置 LOKI_URL 才启用，本机开发与 CI 不受影响 |
| traceId 复用同一来源 | 响应头、访问日志、Loki 里是同一个值，才可能「用户报一次错就定位到机器与耗时」 |




图 6 日志集中化：一台 Loki 收两个实例，按 traceId 一次查全

验证脚本 `scripts\demo-logs.ps1` 六项断言全过：两个实例都在上报、traceId 能精确召回、失败请求可按 traceId 定位、按实例维度可分别检索。

### 7.2 指标与告警

接入 `spring-boot-starter-actuator` 与 `micrometer-registry-prometheus`，暴露 `/actuator/health`（含数据库组件状态）与 `/actuator/prometheus`（HTTP QPS/RT、JVM 堆与线程、磁盘）。指标统一带 `application=campus-shop` 标签，可直接被 Prometheus 抓取。

GET /actuator/prometheus —— 指标暴露

```
application_ready_time_seconds{application="campus-shop"
disk_free_bytes{application="campus-shop",path="C:\\User
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
```

图 7　/actuator/prometheus 实际输出（已产生真实流量）

告警规则见 `docs/ops/alert-rules.yml`：后端不可达、健康检查异常、5xx 错误率 > 1%、P95 > 500ms、堆使用率 > 90%、磁盘 < 10%。原则是只对用户可感知的症状告警，不对单条慢 SQL 告警，避免告警疲劳。

### 7.3 多实例与高可用

限流器原本是进程内滑动窗口：单实例没问题，一旦扩到 N 个副本，每个副本各算各的，**实际放行量被放大 N 倍，限流等于没做**。现已提供 Redis 实现（ZSET + Lua 脚本，清理过期成员、计数、写入三步原子完成），通过 `campus.ratelimit.store=redis` 切换，对外接口不变。

多实例：Redis 共享限流 + 实例级 HTTPS

```
=== A. 两个实例交替请求，限流配额全局共享 ===
压测目标：同一份配额，两个实例交替打
限流规则：/api/v1/auth/refresh  10 次 / 60 秒 / IP (Redis
第  1 次  8080 http   -> 401  <- 放行 (令牌无效)
第  2 次  8443 https  -> 401  <- 放行 (令牌无效)
第  3 次  8080 http   -> 401  <- 放行 (令牌无效)
第  4 次  8443 https  -> 401  <- 放行 (令牌无效)
第  5 次  8080 http   -> 401  <- 放行 (令牌无效)
第  6 次  8443 https  -> 401  <- 放行 (令牌无效)
第  7 次  8080 http   -> 401  <- 放行 (令牌无效)
第  8 次  8443 https  -> 401  <- 放行 (令牌无效)
第  9 次  8080 http   -> 401  <- 放行 (令牌无效)
第 10 次  8443 https  -> 401  <- 放行 (令牌无效)
第 11 次  8080 http   -> 429  <- 限流拦截
第 12 次  8443 https  -> 429  <- 限流拦截
第 13 次  8080 http   -> 429  <- 限流拦截
结论：10 次后两个实例同时开始拒绝 —— 配额是全局共享的，不是每实例
=== B. 第二个实例跑在 HTTPS 上 ===
连接：https://localhost:8443   (TLS 握手成功，自签证书)
Subject : CN=localhost
Issuer  : CN=localhost
有效期  : 2028/10/25 12:29:17
算法    :
HTTP    : 200
```

改动前限流计数在进程内，2 个副本 = 配额被放大 2 倍。改为 Redis ZSET + Lua 后

图 8　两个实例交替请求：10 次后同时开始拒绝，配额全局共享；右侧为 8443 的 TLS 连接

同时启用实例级 HTTPS：`scripts/gen-cert.ps1` 生成自签 PKCS12，`application-https.yml` 指定端口与密钥库，实测 `https://localhost:8443` 返回 200。生产建议在 Nginx / 云负载均衡终止 TLS，应用保持内网 HTTP。

> 一致性取舍：Redis 抖动时限流器 **fail-open**（放行并打 WARN），因为限流是保护手段，不该因缓存故障把业务打死。这个取舍写在代码注释里。

## 7.4 CI/CD 流水线

| 作业 | 关键步骤 |
| --- | --- |
| 后端编译与测试 | JDK 17 → 38 用例 → OpenAPI 契约漂移校验 → **覆盖率门禁（≥35%）** → 上传 surefire 与覆盖率产物 |
| 前端构建与测试 | Node 20 → 19 用例 → 构建 → **依赖漏洞门禁（生产依赖 high/critical）** → 构建镜像 |
| 容器编排验证 | compose 语法校验 → up --build --wait 起四服务 → 经 Nginx 反代冒烟 → 校验种子数据 → 收摊 |

依赖巡检用 Dependabot 每周开 PR（maven / npm / actions），不把扫描塞进构建链路——Trivy 类工具需要拉漏洞库，在受限网络下会把构建拖到十几分钟。



图 9　GitHub Actions 运行与作业明细

## 八、业务闭环演示

下面的输出由 `scripts/demo-business-loop.ps1` 真实调用接口产生，没有 mock、没有直接改库。

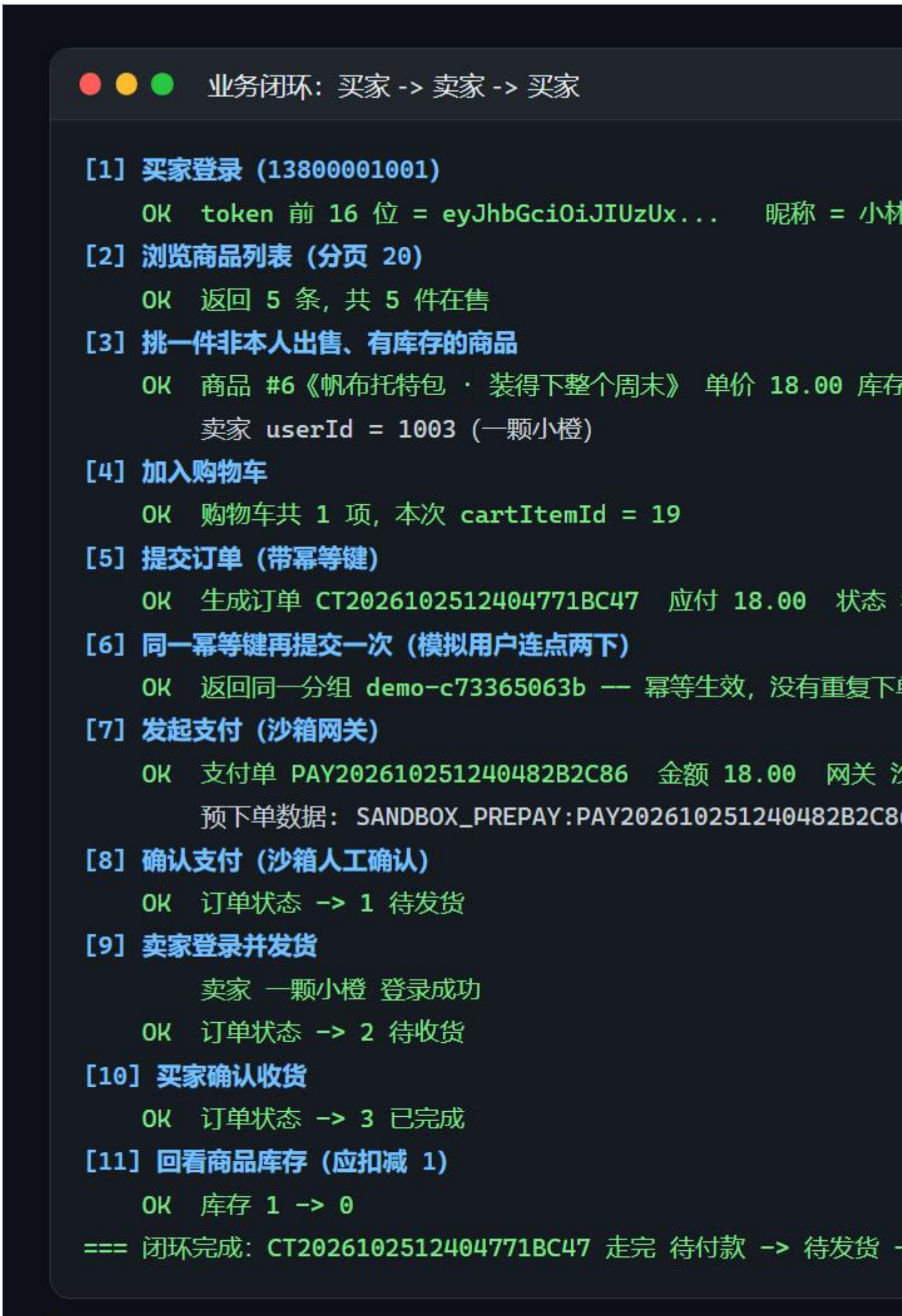

业务闭环：买家 -> 卖家 -> 买家
[1] 买家登录（13800001001）
OK token 前 16 位 = eyJhbGciOiJIUzUx...
[2] 浏览商品列表（分页 20）
OK 返回 5 条，共 5 件在售
[3] 挑一件非本人出售、有库存的商品
OK 商品 #6《帆布托特包 · 装得下整个周末》 单价 18.00
卖家 userId = 1003（一颗小橙）
[4] 加入购物车
OK 购物车共 1 项，本次 cartItemId = 19
[5] 提交订单（带幂等键）
OK 生成订单 CT20261025124047 71BC47 应付 18.00
[6] 同一幂等键再提交一次（模拟用户连点两下）
OK 返回同一分组 demo-c73365063b —— 幂等生效，没有重复下
[7] 发起支付（沙箱网关）
OK 支付单 PAY20261025124048 2B2C86 金额 18.00
预下单数据：SANDBOX_PREPAY:PAY20261025124048 2B2C8
[8] 确认支付（沙箱人工确认）
OK 订单状态 -> 1 待发货
[9] 卖家登录并发货
卖家 一颗小橙 登录成功
OK 订单状态 -> 2 待收货
[10] 买家确认收货
OK 订单状态 -> 3 已完成
[11] 回看商品库存（应扣减 1）
OK 库存 1 -> 0
=== 闭环完成：CT20261025124047 71BC47 走完 待付款 -> 待发货

图 10　买家下单到确认收货的完整闭环

| 步骤 | 动作 | 验证点 |
| --- | --- | --- |
| 1-3 | 买家登录 → 浏览列表 → 选品 | JWT 签发；分页与在售过滤；排除自售商品 |
| 4 | 加入购物车 | 同商品合并，返回 cartItemId |
| 5-6 | 提交订单 → 同幂等键再提交 | **两次返回同一 groupNo**，没有重复下单 |
| 7-8 | 创建支付 → 沙箱确认 | 支付单号、预下单数据；订单转入待发货 |
| 9 | 卖家登录 → 发货 | 只有卖家能发货；订单转入待收货 |
| 10-11 | 买家确认收货 → 回看库存 | 状态转已完成；库存 1 → 0 并置为售罄 |

> 闭环里最关键的两点是：幂等（连点不会出两单）和库存（下单即原子扣减，失败回滚）。两者都有并发用例覆盖，也在这次演示中肉眼可见。

## 九、角色权限演示

同一批接口，匿名、买家、卖家三种身份得到三种结果。输出由 `scripts/demo-rbac.ps1` 真实产生。

角色权限：匿名 / 买家 / 卖家

```
=== 角色权限演示：同一接口，三种身份 ===
[A] 未登录访问受保护接口（HTTP 401）
  GET  /api/v1/orders（匿名）                    -> 401
  POST /api/v1/orders（匿名）                    -> 401
  GET  /api/v1/orders/sold（匿名）               -> 401
  买家 = 小林同学（id 1001）    卖家 = 一颗小橙（id 1003）
[B] 数据隔离：买家看到「我买的」，卖家看到「我卖的」
  GET /api/v1/orders（买家）                     -> 200   {
  GET /api/v1/orders/sold（买家，应为空）          -> 200
  GET /api/v1/orders/sold（卖家，应有数据）        -> 200
[C] 越权边界：买家不能替卖家发货
  演示订单 CT20261025124523D7BFA2（已支付，待发货）
  PUT /orders/{no}/ship（买家身份，越权）          -> 404
  PUT /orders/{no}/ship（卖家身份，合法）          -> 200
[D] 探针端点保持匿名可访问（负载均衡 / K8s / Prometheus 用）
  GET /actuator/health（匿名）                   -> 200   {
  GET /actuator/prometheus（匿名，仅暴露指标）     -> 200
serv...
=== 结论：认证 401、越权 404（不泄露资源存在性）、数据按角色隔离
```

认证失败必须是 HTTP 401（本次顺带修掉了「只写 body code、HTTP 回 200」的

图 11　认证、数据隔离与越权拦截

| 请求 | 身份 | 结果 | 含义 |
| --- | --- | --- | --- |
| GET /api/v1/orders | 匿名 | 401 | 没有凭证，直接拒绝 |
| GET /api/v1/orders | 买家 | 200 | 只返回自己作为买家的订单 |
| GET /api/v1/orders/sold | 买家 | 200 | 返回空数组——没有卖出过东西 |
| GET /api/v1/orders/sold | 卖家 | 200 | 返回自己作为卖家的订单 |
| PUT /orders/{no}/ship | 买家（越权） | 404 | 按「订单不存在」处理，不泄露资源是否存在 |
| PUT /orders/{no}/ship | 卖家（合法） | 200 | 状态机允许，发货成功 |
| GET /actuator/health | 匿名 | 200 | 探针必须匿名，否则负载均衡无法判断存活 |

**本次顺带修掉的一个真问题：**认证失败原先只写响应体里的 `code:401`，HTTP 状态却是 200。网关、CDN、监控和前端拦截器都按 HTTP 状态判断成败，这会让「未登录」被当成成功请求；前端也只有拿到 401 才会走刷新令牌分支。根因是 `ServletUtils.renderString` 里写死了 `setStatus(200)`，已增加带状态码的重载并修正入口点。

## 十、关键难点

| 难点 | 问题 | 做法与证据 |
| --- | --- | --- |
| 库存防超卖 | 「先查再改」在并发下必超卖 | 条件原子扣减：`update ... set stock = stock - #{qty} ... where stock >= #{qty}`，由数据库保证；注意 MySQL 的 SET 从左到右求值，售罄判断必须写在扣减之前，否则剩 1 件会被误判售罄。StockConcurrencyTest 并发验证 |
| 重复下单 | 用户连点、网络重试都会产生两张单 | 幂等键 + `ct_idempotency` 唯一键 + `uk_ct_order_idem`，重复提交返回同一 groupNo；演示与 IdempotencyConcurrencyTest 双向验证 |
| 多卖家拆单 | 一个购物车可能来自多个卖家，金额与运费要按卖家拆 | `uk_ct_order_group_seller` 保证同组同卖家唯一；拆单与库存扣减在同一事务，任一失败整体回滚 |
| 多实例限流失效 | 进程内计数在 N 个副本下被放大 N 倍 | Redis ZSET + Lua 原子计数，两个实例共享一份配额；Redis 故障 fail-open 并告警 |
| HTTP 状态码语义 | 认证失败返回 HTTP 200 + body code 401 | 根因是 renderString 写死 setStatus(200)；新增带状态码重载，认证失败回 401，越权回 404 |
| JaCoCo 中文路径 | 仓库放在中文目录时 agent 静默不产出 exec，覆盖率恒为 0 | agent 通过命令行接收输出路径，中文被错误解码；`scripts/coverage.psl` 自动复制到英文临时目录运行，CI 本身是英文路径不受影响 |
| 备份可恢复性 | dump 文件存在 ≠ 能恢复 | mysqldump 默认带 GTID_PURGED，恢复回本机必然冲突；加 `--set-gtid-purged=OFF` 并做往返验证：45 表 / 8 商品 / 2 订单完全一致 |

# 十一、AI 使用与复现

## 11.1 AI 用在哪些环节

| 环节 | 具体用法 | 人工把关 |
| --- | --- | --- |
| 需求拆解 | 把「交易闭环」拆成状态机与接口清单 | 状态跃迁是否合法逐个人工确认 |
| 编码 | 生成 Controller / Service / Mapper 骨架与 DTO | 分层边界、事务范围人工复核 |
| 测试 | 补并发、幂等、越权类用例 | 断言必须来自业务规则，不接受「跑通即对」 |
| 排障 | JaCoCo 覆盖率恒为 0、dump 无法恢复、认证返回 200 | 结论必须能被一次实验复现才算数 |
| 文档 | 整理能力对照、压测与覆盖率报告 | 所有数字必须来自本机实际执行 |

> 使用边界：AI 给的是候选方案，凡是「看起来能跑」的结论都要落到可复现的实验上。本次三个真问题（覆盖率、GTID、401）都是先怀疑 AI 的结论、再做对照实验定位的。文档里没有未经实测的性能或覆盖率数字。

## 11.2 复现路径

```
启动后端.cmd                          检查 MySQL/Redis → 定位 java/mvn → 构建并启动
启动前端.cmd                          缺依赖时自动 npm install
检查服务.cmd                          后端/前端/文档/代理链路逐项自检
docs\采集环境信息.cmd                 打印并生成运行环境版本图
scripts\coverage.ps1                  JaCoCo 覆盖率（自动规避中文路径）
scripts\check-coverage.ps1            覆盖率门禁（默认阈值 35%）
scripts\loadtest.mjs                  压测：node scripts\loadtest.mjs http://localhost:8080 20 20
scripts\demo-business-loop.ps1        业务闭环演示
scripts\demo-rbac.ps1                 角色权限演示
scripts\gen-cert.ps1                  生成本地 HTTPS 自签证书
scripts\backup.ps1 / restore.ps1      数据库备份与恢复
```

## 11.3 测试账号

| 角色 | 手机号 | 密码 |
| --- | --- | --- |
| 买家示例 | 13800001001 | admin123 |
| 卖家示例 | 13800001003 | admin123 |

## 十二、接口文档与运行界面



图 12　Swagger UI 接口文档



图 13　OpenAPI 3 契约文档 /v3/api-docs/campus



图 14　Druid 数据库监控台



图 15　健康检查接口返回



图 16　前端首页（经代理读取后端数据）

# 十三、总结

## 13.1 做完了什么

- 交易闭环可用：浏览 → 收藏 → 购物车 → 下单（多卖家拆单 + 幂等）→ 支付 → 发货 → 收货 → 退款，状态机在服务端强制
- 数据正确性有保障：原子扣减防超卖、幂等键防重复、事务覆盖拆单与扣减、10+ 唯一索引兜底
- 工程化闭环：38 + 19 个用例、覆盖率 43.2% 且有 35% 门禁、OpenAPI 契约防漂移、CI 三作业 + 依赖巡检
- 可运维：指标与 6 条告警规则且已打通 Alertmanager→webhook 投递、健康检查、TraceId 全链路、备份已注册每日计划任务并做过恢复往返验证
- 可扩展：限流已支持 Redis 共享计数，多实例不再是「限流失效」

## 13.2 数据说话

| 指标 | 数值 | 口径 |
| --- | --- | --- |
| 单机压测 QPS | 1354（并发 50） | 4 个读接口加权混合，同机自压 |
| P95 延迟 | 56.8ms | 同上 |
| 错误率 | 0.00% | 49499 个请求零失败 |
| 业务模块行覆盖 | 43.2% | JaCoCo，含 73 个类 |
| 自动化用例 | 57 | 后端 38 + 前端 19 |

## 13.3 还差什么

距企业级互联网应用标准，业务代码与工程习惯已经够用，差的是「生产运行时」这层：多实例与负载均衡、数据库主从与故障切换、日志集中化（ELK/Loki）、熔断降级、灰度发布、静态检查与 E2E。这些在能力对照表里都标了「未覆盖」，没有包装成已完成。

建议的补齐顺序：① 多实例 + LB（限流与定时任务已就绪）② 日志集中化 ③ 灰度与回滚演练 ④ 静态检查与 E2E ⑤ 有真实异步需求时再上 MQ。

## 13.4 一句话

> 这套系统的价值不在于用了多少组件，而在于每个关键结论都能被一次实验复现：超卖有并发用例、幂等有重复提交、覆盖率有报告、限流有双实例对照、备份有往返恢复、性能有实测数字。

# 附录　答辩问答稿（10 问）

> 每问的答法：先给结论，再说理由，再指证据，最后讲边界。完整版见 docs/答辩问答稿.md。

**Q1　为什么不做微服务？单体怎么保证可扩展？**
交易、库存、支付共享一个库，拆开就要处理分布式事务，现阶段是负收益。可扩展性放在了真正先出问题的地方：限流器支持 Redis 共享计数（多副本不再失效）、无状态 JWT（会话不绑实例）。证据：CtOrderServiceImpl:120,327 的事务边界、SlidingWindowLimiter 双实现。边界：日订单到十万级或需独立扩缩不同域时，先拆「支付回调 + 通知」这类异步链路，而不是核心交易。

**Q2　库存怎么防超卖？为什么不用 version 乐观锁？**
用条件原子扣减，把判断和扣减压进一条 SQL：`update ... set stock = stock - ? where stock >= ?`，affectedRows=0 即库存不足。乐观锁要两次往返且失败要重试，徒增复杂度。踩过的坑：MySQL 的 SET 从左到右求值，售罄判断必须写在扣减之前，否则剩 1 件会被误判售罄。证据：CtProductMapper.xml 的 deductStock 及其注释、StockConcurrencyTest。边界：分库分表后同商品库存必须路由到同一分片。

**Q3　幂等怎么做的？为什么幂等表要和订单表分开？**
客户端带 idempotencyKey，靠 ct_idempotency 的唯一键由数据库裁决，重复提交返回同一 groupNo。「先查订单表看有没有」在并发下不安全——两个请求会同时查到没有。分开建表是因为幂等键语义更长：一次请求可能产生多张订单（拆单），也可能什么都没产生。证据：uk_ct_idem_key、IdempotencyConcurrencyTest、闭环演示连点两次返回同一 groupNo。

**Q4　多卖家拆单，一个卖家成功一个失败怎么办？**
拆单、扣库存、写幂等记录在同一个事务里，任一失败整体回滚，不存在「一半成功」。跨库跨服务才需要 Saga 补偿。证据：uk_ct_order_group_seller 唯一索引、OrderLifecycleTest。边界：若把支付独立出去，需本地消息表 + 补偿任务。

**Q5　限流为什么用滑动窗口？多实例为什么必须 Redis？Redis 挂了怎么办？**
固定窗口有临界点双倍流量问题（59 秒与 61 秒各发 30 个即打穿）；进程内计数在 N 个副本下被放大 N 倍，是功能性失效而非性能问题。故用 ZSET + Lua 原子计数。Redis 故障时 fail-open（放行 + WARN），因为限流是保护手段，不该把业务打死。证据：RedisSharedRateLimitTest 的两实例共享与内存对照组、实测第 11 次起两个实例同时 429。

**Q6　「认证失败返回 HTTP 200」怎么发现的？为什么之前没发现？**
写角色权限演示用 curl 看状态码时发现：body 写着 code:401，HTTP 却是 200。之前没发现是因为前端只读 body 的 code，UI 表现一致；单测断言的是业务码。危害：网关与监控按 HTTP 状态判成败，错误率指标永远正常；前端判断 `error.status === 401`，刷新令牌分支永不触发。根因是若依 ServletUtils.renderString 里写死 setStatus(200)，已加带状态码重载。

**Q7　覆盖率只有 43%，是不是测试不够？**
43% 是全部 73 个业务类口径；核心路径远高于此（CtOrderServiceImpl 85.7%），拉低整体的是 DTO 与实体。我做了两件事而不是只解释：CI 加 35% 门禁防退化；门禁失败时列出未覆盖行最多的 10 个类，指哪补哪。证据：check-coverage.ps1 正反两向实测。

**Q8　1354 QPS 是同机自压，怎么证明真实能力？**
同机自压只能说明代码不是瓶颈。观察：并发 20→50，QPS 只涨 21% 而 RT 涨一倍，是服务端饱和的典型信号；健康检查 4ms 说明框架开销很小；49499 请求零错误，说明饱和时是排队而不是报错。瓶颈在同机 CPU 争抢与连接池 max-active=8。要拿真实容量需独立压测机，我没做，也不宣称。

**Q9　离企业级上线还差什么？两周先做什么？**
差「生产运行时」这层，但最近一轮把三件「看起来做了其实没生效」的事补成了真的：定时任务已切 JDBC 集群（双实例只跑一次）、备份已注册每日计划任务并用真实触发验证、告警已补上健康指标并打通 Alertmanager 投递。剩下优先级：①多实例 + LB ②日志集中化 ③灰度与回滚演练 ④静态检查与 E2E。先不做 MQ / 分布式事务 / CDN。证据：企业级能力对照表的「未覆盖项汇总」表。

**Q10　AI 用到什么程度？怎么保证 AI 写的代码是对的？**
用在骨架生成、测试补全、排障推理、文档整理；每个结论必须落到一次可复现实验上。本次三个问题

（覆盖率恒为 0、dump 无法恢复、认证返回 200）都是先怀疑结论再做对照实验定位的——例如把 JaCoCo 输出改到纯英文目录 C:\jacoco-test，文件立刻产出，才确认是中文路径问题。判断标准只有一条：能不能用一次实验证伪它。

## 三个兜底回答

| 问题 | 诚实回答 |
| --- | --- |
| 为什么没有 XX 组件？ | 不需要，而且我能说清为什么不需要——单库单体上 MQ/分布式事务是负资产 |
| 覆盖率为什么不是 80%？ | 核心路径 85%，整体被 DTO 拉低；有 35% 门禁防退化，提到 70% 需补 Controller 层用例 |
| 做过什么事故复盘？ | 没有线上环境，但本机复现并修复了 3 个真实缺陷：中文路径致覆盖率失效、GTID 致备份无法恢复、HTTP 200 掩盖认证失败 |

# 附录 B　三件「看起来做了但没生效」的事

> 这三项的共同点：配置或脚本都写了，但运行时等于没做。判断标准很简单 —— **能不能拿出一条本次执行产生的证据**。

## B.1 定时任务：从 RAMJobStore 到真正集群

**问题：**启动日志一直是 `Using job-store 'org.quartz.simpl.RAMJobStore' - which does not support persistence. and is not clustered.`。这种状态下起两个实例，三个默认任务会被两个节点各跑一遍，日志双写、业务重复执行。

**做法：**切 `spring.quartz.job-store-type=jdbc`；表交给 Flyway（`V3__quartz_cluster_tables.sql`，全部 `create table if not exists`），并设 `initialize-schema: never`，避免 Spring 每次启动把调度状态和锁表 DROP 掉。同时改造 `SysJobServiceImpl.init()`：原来是 `scheduler.clear()` + 全量重建，后启动的节点会把已在跑的触发器删掉；现在先查 `QRTZ_SCHEDULER_STATE`，60 秒内有其他节点心跳就跳过初始化。

**证据：**两个实例启动后 `QRTZ_SCHEDULER_STATE` 有 2 行；节点 B 日志出现「集群模式下已有其他调度实例在线，跳过任务初始化」；任务执行记录严格 10 秒一条、没有同秒成对：

```
18:53:10  18:53:20  18:53:30  18:53:40  18:53:50  18:54:00   （sys_job_log，cron 0/10 * * * * ?）
```

## B.2 备份：脚本存在 ≠ 备份在跑

**问题：**`backup.ps1` 早就写好并做过恢复验证，但没有任何调度 —— 没有人会每天手动点一下。而且计划任务没有交互环境变量，直接注册必然取不到数据库密码。

**做法：**新增 `scripts/install-backup-task.ps1`：口令写入 `%LOCALAPPDATA%\campus-shop\db-password` 并用 icacls 限定当前用户；用 `ScheduledTasks` 模块注册每日 02:00 的任务（不用 schtasks 拼引号，踩过两次）；装完立刻 `Start-ScheduledTask` 验证真能产出文件。备份脚本本身补了三件事：写 `logs/backup.log`、文件小于 1KB 判失败、支持 `-Offsite` 异地副本。

**证据：**LastTaskResult = 0；新备份 `shop-20261025-185836.sql` 130.5 KB；异地目录同样落盘；日志里记录了 SHA256 与保留份数。

## B.3 告警：规则写了，但指标根本不存在

**问题：**`HealthDegraded` 规则写的是 `health{job="campus-shop"} < 1`。但 `/actuator/health` 是**端点**，不是指标，Micrometer 里没有 `health` 这个名字。Prometheus 会得到空结果集，规则永远不触发，而且不会有任何报错。

**做法：**补一个 `campus_health_status` gauge（1=UP，0=非 UP，由 `HealthEndpoint` 实时取值），规则改用它；再用 `scripts/check-alerting.ps1` 把这件事变成可回归的检查 —— 抽取规则里引用的每个指标名，去实时 `/actuator/prometheus` 里比对，不存在就失败。

**证据：**自检第一次跑就报出 3 个不存在的指标（`health`、`http_server_requests_seconds_count`、`_bucket`），其中后两个是「刚启动还没有请求序列」的假阳性，已通过预热 + 解析 `# TYPE` 派生修正；修完 `campus_health_status` 后自检全绿。

接收端也做了真实验证：本地起 Alertmanager，推一条 `severity=critical` 的测试告警，被路由到 `critical-webhook` 并投递成功，webhook 收到 883 字符载荷：

```
{"receiver":"critical-
webhook","status":"firing","alerts":[{"labels":{"alertname":"CampusDemoAlert4","severity":"critical"}}]}
```